

Emotion Classification Using LSTM on Twitter Data

James Bayley

University of Essex

jb22375@essex.ac.uk

Abstract

This report presents a methodology for emotion classification using a Sequential LSTM model. The dataset comprises over 400,000 tweets labelled with six emotions: sadness, joy, love, fear, anger and surprise. Preprocessing steps included normalisation, stop word and punctuation removal and tokenisation into sequences of 100 tokens. The Sequential LSTM achieved a mean accuracy of 93.7% and a percentage of mean accuracy of 8% suggesting a moderately robust model.

1 Introduction

Emotion classification is essential for understanding user sentiment in posts on social media. This could be used to gauge public opinion or identifying users in emotional distress. [1] Text classification can be used to detect spam/junk mail, categorise emails, classify reviews, categorise products and news among many other applications.

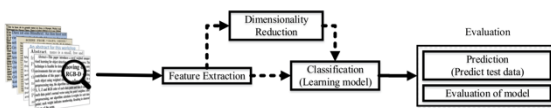


Figure 1. Diagram of text classification pipeline.

Most text classification approaches follow a four-phase classification pipeline. This starts with feature extraction, where unstructured text is taken and converted into a structured feature space when using mathematical modelling in the classifier. This step includes removing stop words, punctuation and other unnecessary words and/or characters. Dimension reduction is used to reduce memory and time complexity of an application, instead of using inexpensive classifiers which tend not to perform well for some datasets. In my case, I did not use this approach. The third step of the pipeline is the classification technique (learning model). There are many different classification techniques which exist, and all have their own advantages and disadvantages for any given application. The final step is evaluation, where a model's effectiveness is measured. Accuracy measurement is the simplest method for evaluation but does not work for unbalanced data. [2]

2 Methodology

2.1 Data Preprocessing

Data preprocessing is an important step in text classification. Initially, all characters are converted to lowercase for uniformity across the dataset. Stop words were removed since they are semantically insignificant in many contexts. Their presence in text can confuse a machine learning algorithm [3]. Punctuation was also removed since its semantic role is often negligible in similar contexts. [4].

Using the train test split function from Scikit-Learn's Model Selection library [5], the partially pre-processed data is split into training and testing data sets, with a 90-10 train-test split. A Tokenizer from TensorFlow's Keras library [6] was then used to convert the pre-processed text into sequences of integers, with a vocabulary size capped at 10,000 to focus on the most frequent terms. The sequences were padded to a fixed length of 100 tokens to ensure uniformity across all samples.

2.2 Classification Model

Understanding the dataset was key to choosing the correct classification method to use. The dataset used in this classification contain over 400,000 tweets each with an emotion label (0 – sadness, 1 – joy, 2 – love, 3 – anger, 4 – fear, 5 – surprise). [7] The relationships between the labels and the tweets are non-linear, meaning classifiers like a Linear Support Vector Machines would not be appropriate.

Based on empirical evidence gathered during experimentation, it was determined that the Sequential LSTM model is the most suitable method for my objective. Recursive Neural Networks, and in particular Long Short-Term Memory networks have been shown to be far more effective over other classification methods in regard to text classification and capturing sentiment within them. [8] Initially, a non-linear Support Vector Machine (SVM) with Radial Basis Function (RBF) kernel was tested. This method was found to be unsuitable since it failed to capture the dataset's semantic and sequential complexity. Because of its

inability to handle the sparse, non-linear relationships, the model failed to converge and proved computationally too expensive.

2.3 Model Architecture

The Sequential LSTM used was constructed using TensorFlow’s Keras API [6] with the following architecture.

Embedding Layer – The first layer of the model is the embedding layer, which converts the input integer into sequences into dense vectors of fixed size. Dense vectors are effective in NLP applications due to their ability to capture semantics and encode contextual relationships. Vocabulary size was limited previously to 10,000 and represented as a 100-dimensional vector.

LSTM Layer – The second layer is a Long Short-Term Memory network with 128 units. Return sequences is set to false since only the final hidden state is sufficient for our classification as each sequence corresponds to a single emotion label.

Dropout Layer – Dropout is a regularisation technique where randomly selected units are ignored in training to force the model to learn more robust representations. Overfitting is when a machine learning algorithm fits well or perfectly to its training data but can’t make accurate prediction on any data aside from its training data. [10]

Dense Layers – In this implementation, there are two dense layers. The first layer has 64 units and uses the Rectified Linear Unit (ReLU) activation function. ReLU activation adds linearity into the model, allowing it to learn complex patterns in the data. The second layer has 6 units (for the 6 emotion categories) and adds a Softmax activation function, which is ideal for multiclass classification tasks.

2.4 Compilation and Training

Once the architecture was defined, the model was compiled with the following configurations:

The Adam optimiser was used, known for its adaptive learning rates and efficiency in training deep learning models. Since the problem is a multiclass classification problem, sparse categorical cross-entropy was chosen as the loss function. This is appropriate when the labels are integers. The accuracy metric was chosen to evaluate the performance of the model during training and testing. The model is trained for 4 epochs with a batch size of 256 and a validation split of 0.2

was used to monitor the model’s performance on unseen data during training. 4 epochs were chosen since running any additional epochs after the fourth no longer has any benefit in increasing accuracy or reducing loss (see Table 1).

Epoch	accuracy	loss
1	0.6942	0.7752
2	0.9376	0.1103
3	0.9412	0.0935
4	0.9433	0.0879
5	0.9426	0.0889
6	0.9441	0.0848

Table 1. Accuracy and loss by epoch number

3 Results

The performance of the model was evaluated using accuracy, precision, recall and F1-score metrics along with a confusion matrix for visualisation of test classification (see Figure 2).

The final test results came out at 0.937 accuracy and 0.098 loss on the test set. The results of the classification report are shown in Table 2.

	precision	recall	f1-score
sadness	0.97	0.98	0.97
joy	0.93	0.99	0.96
love	1	0.7	0.96
anger	0.91	0.96	0.94
fear	0.91	0.88	0.9
surprise	0.84	0.78	0.81
accuracy			0.94
macro avg.	0.93	0.88	0.9
weighted avg.	0.94	0.94	0.93

Table 2. Classification report of test set

The confusion matrix (see Figure 2) shows that model struggled with identifying the emotion love effectively, instead confusing it with joy. This may occur

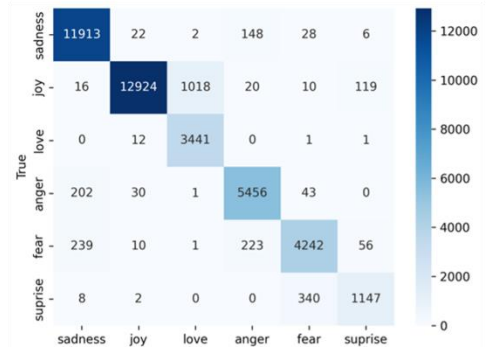


Figure 2. Confusion Matrix of emotion classes

since love and joy are both positive emotions which can be expressed similarly, and often at the same time. The matrix also displays an imbalance between the number of samples in each class, namely surprise and love. This could lead to the model struggling with classifying these classes.

The standard deviation was calculated using:

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

$$\sigma \approx 0.075$$

Where x is equal to the accuracy of each run, μ is equal to the mean accuracy and N is equal to the number of runs.

The percentage mean accuracy can demonstrate how robust our model is. It was calculated as follows:

$$\text{Percentage of Mean Accuracy} = \left(\frac{\text{Standard Deviation}}{\text{Mean Accuracy}} \right) \times 100$$

$$= \left(\frac{0.075}{0.937} \right) \times 100 \approx 8\%$$

4 Conclusion

The Sequential LSTM model proved effective overall with a weight f1-score of 94%, a test accuracy of 93% and a test loss of <1%. However, more tuning of the program could possibly lead to a more effective model. The 8% mean accuracy score suggests that the model's robustness could be improved. In future, improvements could be made by oversampling underrepresented classes, experimenting with attention mechanisms and refining preprocessing steps to capture more informal terms.

Works Cited

- [1] M. Chau, T. M. H. Li, P. W. C. Wong, J. J. Xu, P. S. F. Yip and H. Chen, "FINDING PEOPLE WITH EMOTIONAL DISTRESS IN ONLINE SOCIAL MEDIA: A DESIGN COMBINING MACHINE LEARNING AND RULE-BASED CLASSIFICATION," *MIS quarterly*, vol. 44, no. 2, 2020.
- [2] K. Kowsari, K. Jafari Meimandi, M. Heidarysafa, S. Mendu, L. Barnes and D. Brown, "Text Classification Algorithms: A Survey," *Information*, vol. 10, no. 4, p. 150, 2019.
- [3] S. Alam and N. Yao, "The impact of preprocessing steps on the accuracy of machine learning algorithms in sentiment analysis," *Computational and Mathematical Organization Theory*, vol. 25, pp. 319-335, 2018.
- [4] A. Ek, J.-P. Bernardy and S. Chatzikyriakidis, *How does Punctuation Affect Neural Models in Natural Language Inference*, 2020.
- [5] scikit-learn Developers, "Scikit Learn Model Selection API Reference," Scikit Learn, [Online]. Available: https://scikit-learn.org/stable/api/sklearn.model_selection.html. [Accessed 5 January 2025].
- [6] TensorFlow Developers, "TensorFlow Keras API Documentation," TensorFlow, [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras. [Accessed 5 January 2025].
- [7] N. Elgiriye withana, "Emotions: Where Words Paint the Colors of Feelings," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/nelgiriye withana/emotions>. [Accessed 5 January 2025].
- [8] J. Nowak, A. Taspinar and R. Scherer, "LSTM Recurrent Neural Networks for Short Text and Sentiment Classification," in *Artificial Intelligence and Soft Computing*, Zakopane, 2017.
- [9] IBM, "What is Overfitting?," IBM, 19 December 2024. [Online]. Available: <https://www.ibm.com/think/topics/overfitting>. [Accessed 5 January 2025].
- [10] L.-C. Yu, J. Wang, K. R. Lai and X. Zhang, "Refining Word Embeddings Using Intensity Scores for Sentiment Analysis," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 26, no. 3, pp. 671-681, 2018.